📑 Rapport Technique d'Implémentation & Stratégie de Résilience

Projet : Application web Click & Collect Drive (Appel d'offre Lidl)
Stack Technique : PostgreSQL, Supabase, Prisma, Next.js
Rédigé pour : Chargé cybersécurité, Responsable maintenance, Chef de projet BDD
Date : Avril 2026
Statut : Version Finale — Expertise Production (Budget Illimité)

1. ARCHITECTURE ET MODÉLISATION (SCHÉMA RELATIONNEL)

La base de données est structurée autour de **18 tables actives** réparties en 5 domaines fonctionnels isolés pour garantir l'intégrité et la scalabilité :

- **Utilisateurs & Accès :** `client`, `client_account`, `client_history`, `permission`.
- **Personnel :** `preparer`, `manager`, `schedule`, `performance`.
- **Catalogue & Stock :** `product`, `category`, `stock`.
- **Commandes & Panier :** `order`, `order_item`, `cart`, `cart_item`, `substitution_proposal`, `payment`.
- **Logistique & Magasin :** `store`, `pickup_slot`, `notification`, `audit_log`.

⚖️ Choix d'architecture : Séparation Identité / État

Implémentation du pattern de séparation entre `client` (données PII : identité, contact) et `client_account` (métadonnées d'état : `is_verified`, `is_active`). Cette approche permet le "soft delete", facilite l'anonymisation RGPD sans rupture de l'intégrité référentielle, et isole la gestion de l'authentification.

2. INTÉGRITÉ DES DONNÉES ET GESTION DE LA CONCURRENCE

- **Clés Primaires (PK) :** Standard `bigint GENERATED BY DEFAULT AS IDENTITY` pour lever les conflits avec Prisma.
- **Gestion des Séquences :** Procédure de recalage (`setval`) après chaque import pour prévenir l'erreur d'unicité 23505.
- **Clés Étrangères (FK) :** 100% des relations sécurisées par des contraintes strictes.
- **Concurrence Critique :** Utilisation de **verrous transactionnels** (`SELECT FOR UPDATE`) sur les tables `stock` et `pickup_slot` pour éviter le sur-booking.
- **Contraintes de Check :** `ALTER TABLE stock ADD CONSTRAINT stock_positive CHECK (quantity >= 0);` garantissant l'impossibilité technique d'un stock négatif.

3. STRATÉGIE D'INDEXATION ET OPTIMISATION

Création d'index ciblés pour éliminer les scans de table complets et supporter la charge "Lidl" :

- **Recherche Catalogue :** Index B-Tree sur `product(name)` et Index **GIN** pour la recherche plein texte (Full-Text Search).
- **Requêtes Relationnelles :** Index sur `order(client_id)`, `orderitem(order_id)` et `stock(product_id, store_id)`.
- **Index Partiels :** `CREATE INDEX idx_active_orders ON "order" (id) WHERE status IN ('PENDING', 'PREPARING');` pour accélérer le flux logistique en temps réel.
- **Sécurité :** `CREATE UNIQUE INDEX idx_client_email_unique ON client_account (email);`

4. SÉCURITÉ ET ROW LEVEL SECURITY (RLS)

Activation de la **RLS** sur l'ensemble des tables exposées pour une sécurité "Defense in Depth" :

- **Lecture publique :** Catalogue produits accessible sans authentification (`USING (true)`).
- **Isolation stricte :** Les clients ne peuvent requêter que leurs propres données via `auth.uid() = id`.
- **Privilèges Back-end :** Utilisation de la `Service Role Key` par le serveur Next.js pour les opérations administratives.
- **Schémas :** Séparation logique entre `public` (API), `private` (données sensibles) et `logging` (audit).

5. STOCKAGE DES MÉDIAS (ASSETS)

- **Architecture :** Déploiement d'un bucket Supabase Storage (`images-lidl`) en accès public.
- **Optimisation :** Utilisation d'une colonne `image_url` (type `text`) dans `product` , évitant le stockage binaire (BLOB) pour optimiser les temps de réponse et l'utilisation de la mémoire tampon de PostgreSQL.

6. SAUVEGARDE ET RÉSILIENCE : LE POINT-IN-TIME RECOVERY (PITR)

En contexte de production réelle avec budget illimité, le PITR est l'unique stratégie retenue.

A) Pourquoi le PITR est supérieur aux alternatives

- **Contre le Backup Quotidien :** Le backup quotidien (snapshot) peut engendrer jusqu'à 24h de perte. Le PITR réduit cette perte (RPO) à **moins de 5 secondes**.
- **Contre la Réplication :** Une erreur humaine (`DELETE` accidentel) est répliquée instantanément. Seul le PITR permet de "remonter le temps" avant l'erreur.

B) Mécanisme Technique

Archivage continu des **Write-Ahead Logs (WAL)** vers AWS S3. En cas d'incident, PostgreSQL rejoue les transactions jusqu'au timestamp précis (seconde près) souhaité.

7. CONNECTIVITÉ ET INFRASTRUCTURE DE SCALE

- **Gestionnaire de pool (Supavisor) :** Utilisation obligatoire pour supporter les milliers de connexions simultanées depuis les fonctions Next.js sans saturer la RAM.
- **Variables d'environnement :** `DATABASE_URL` poolée pour Prisma, `SERVICE_ROLE_SECRET` pour les accès élevés.
- **Réplication :** Déploiement de **Read Replicas** géographiques pour réduire la latence sur les requêtes de consultation du catalogue.

8. CONFORMITÉ RGPD ET AUDIT

- **Souveraineté :** Hébergement exclusif en zone **AWS eu-central-1 (Francfort)** pour le maintien des données dans l'EEE.
- **Anonymisation :** Suppression basée sur la pseudonymisation des colonnes PII (Article 17 RGPD), préservant la cohérence des historiques financiers.
- **Traçabilité :** Table `audit_log` enregistrant l'origine (`actor_id`, `ip_address`) et la cible de chaque action critique.
- **Rétention :** 3 ans pour les comptes inactifs et 10 ans pour les données transactionnelles.

9. AUTOMATISATION ET MAINTENANCE

- **Triggers de Maintenance :** Fonctions SQL natives pour la mise à jour automatique de la colonne `updated_at` via `NOW()`.
- **Tâches Planifiées (pg_cron) :** Automatisation du nettoyage des paniers abandonnés (`cart_item`) et des sessions expirées.
- **Monitoring :** Surveillance du flux WAL et de l'occupation disque pour garantir la fenêtre de restauration (7, 14 ou 28 jours).

Conclusion : Cette implémentation garantit à Lidl Drive une résilience totale. Là où un système classique accepte de perdre une journée de commandes, cette architecture assure un retour à la normale en quelques minutes avec une perte de données quasi nulle.